T2112
Développement informatique III (Pratique)

Qualité logicielle & Tests automatisés
TP 04

Auteur(s) :
Jonathan Noël

Dernière mise à jour le
5 mars 2026

Table des matières

1 Introduction **2**

2 Les types de tests - Rappels **2**

3 Mise en place de l'environnement de test - Jest **3**
3.1 Installation 3
3.2 Initialisation de jest.config.js 3

4 Tests via Jest **3**
4.1 Premier test 3

5 Partie 1 : Tests Fonctionnels (Black Box) **4**
5.1 Le Catalog-Based Testing 5
5.1.1 Exercice 5
5.2 Le N-Wise Testing 5
5.3 Conclusion 6

6 Partie 2 : Tests Structurels (White Box) **6**
6.1 La couverture de code 6
6.2 Exercice : Le validateur de mot de passe 7

7 Travail à rendre (À remettre dans 2 semaines) **7**
7.1 Le validateur d'inscription 7
7.2 Livrables attendus 8

Légendes

👨‍🏫 Conseil du professeur.
ℹ️ Une information ...
🚨 Pré-requis pour le TP.
🖥️ Exercice à réaliser.
👓 Exercice d'observation ou de lecture.

1 Introduction

🚨 Pour réaliser ce TP, vous devez absolument avoir terminé le TP précédent. Si ce n'est pas le cas, c'est que vous êtes en retard et que vous devriez le rattraper au plus vite. Attention que tous les TP suivants auront comme prérequis le précédent.

Jusqu'à présent, pour vérifier si votre application fonctionnait, vous lanciez le serveur, ouvriez votre navigateur, et vous cliquiez manuellement. C'est ce qu'on appelle du "test manuel". Le problème ? C'est lent, répétitif et sujet à l'erreur humaine. Si vous modifiez une fonction dans 6 mois, allez-vous penser à retester manuellement toutes les routes ? Probablement pas.

Comme vu au cours (P5.1), l'automatisation des tests permet de garantir la fiabilité et la maintenabilité du code. Dans ce TP, nous allons écrire des scripts qui testeront votre code à votre place. Tester ne consiste pas seulement à vérifier que "ça marche" dans le meilleur des cas. Tester, c'est détruire votre application de manière méthodique pour garantir qu'elle survit aux erreurs, aux cas limites et aux combinaisons improbables.

Dans ce TP, nous allons explorer deux approches complémentaires :

— **Test Fonctionnel (Black Box)** : On teste à partir des spécifications, sans regarder le code. On utilise des méthodes comme le Catalog Based Testing et le N-Wise.

— **Test Structurel (White Box)** : On teste à partir du code source, en cherchant à exécuter chaque ligne et chaque branche conditionnelle (Coverage).

Pour certaines séances de travail pratique, il est interdit d'utiliser les outils d'intelligence artificielle génératives afin que vous appreniez les fondamentaux correctement, ce qui est le cas de cette seconde séance. Aidez-vous un maximum des documentations officielles liées aux technologies que vous utiliserez dans le tp. Si dans les 30 dernières minutes du TP vous n'arrivez pas à avancer, aidez-vous de l'IAg, mais pas avant !

2 Les types de tests - Rappels

Nous allons nous concentrer sur deux niveaux de la pyramide de tests :

— **Tests unitaires** : Vérifier qu'une fonction isolée (ex : un calcul, une validation) renvoie bien le résultat attendu pour une entrée donnée.

— **Tests d'intégration** : Vérifier que l'ensemble de la chaîne (Route -> Controller -> [Service] -> Réponse) fonctionne correctement en simulant des requêtes HTTP.

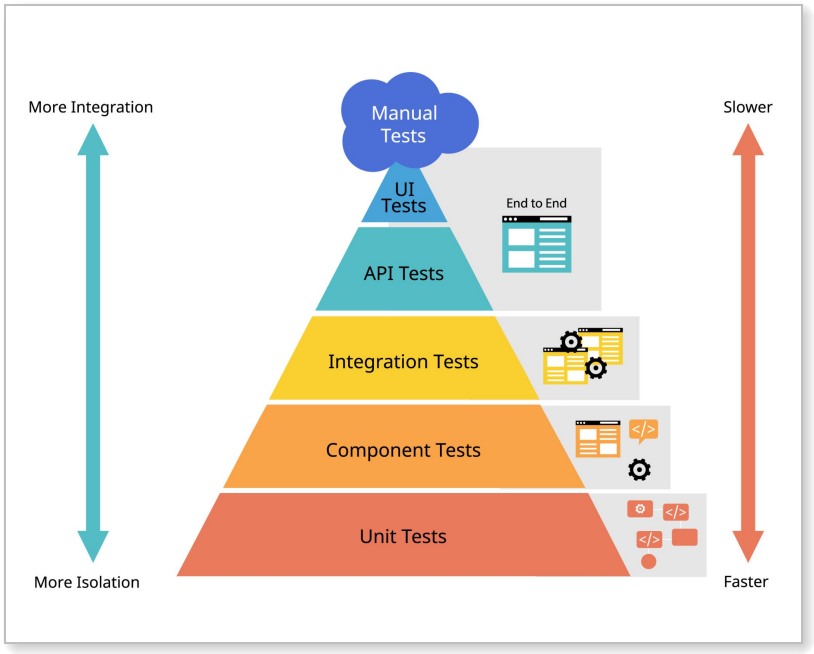

FIGURE 1 – Pyramide de tests en qualité logicielle

3 Mise en place de l'environnement de test - Jest

Nous allons utiliser Jest, le framework de test le plus populaire pour JavaScript/TypeScript.

3.1 Installation

Nous testerons principalement le backend, c.à.d. le projet Express. Placez-vous dans le dossier server et installez la librairie de test.

```
npm install jest ts-jest @types/jest supertest @types/supertest --save-dev
```

- **jest** : Le lanceur de tests.
- **ts-jest** : Un préprocesseur pour permettre à Jest de comprendre TypeScript.
- **supertest** : Une librairie pour simuler des requêtes HTTP vers le serveur Express sans avoir besoin de le lancer manuellement.

3.2 Initialisation de jest.config.js

Il est nécessaire d'initialiser un fichier de configuration `"jest.config.js"` afin que jest puisse fonctionner correctement. Dans un premier temps, générons le fichier grâce à la commande

```
npx ts-jest config:init
```

Ensuite, ajoutez les scripts suivants dans le package.json.

```
"scripts": {
    "test": "jest",
    "test:watch": "jest --watch",
    "test:coverage": "jest --coverage"
}
```

4 Tests via Jest

Un test se compose généralement de trois parties (Pattern AAA) :

- **Arrange** : Préparer les données c.à.d. les valeurs possibles que chaque paramètre peut prendre.
- **Act** : Exécuter la fonction à tester avec les valeurs sélectionnées.
- **Assert** : Vérifier que le résultat est celui attendu (expect).

4.1 Premier test

Créez un dossier `"src/tests"` et un fichier `"math.test.ts"`. Dans ce fichier, ajoutez le code suivant :

```
// Fonction à tester (normalement importée depuis votre code)
const add = (a: number, b: number) => a + b;

describe('Math Utils', () => {
    it('devrait additionner deux nombres correctement', () => {
        const result = add(2, 3);
        expect(result).toBe(5);
    });

    it('devrait gérer les nombres négatifs', () => {
        expect(add(-1, -1)).toBe(-2);
    });
});
```

Lancez `"npm test"` pour voir le résultat. Tous les fichiers comprenant `".test"` dans leur nom vont être pris en compte lors de l'exécution des tests.

👨‍🏫 Que se passe-t-il ? Comprenez bien les output de la commande.

5 Partie 1 : Tests Fonctionnels (Black Box)

ℹ️ Mais comment réellement décider des données de tests à couvrir ?

Dans cette partie, vous ne voyez pas le code de la fonction. Vous ne disposez que de ses spécifications. Premièrement, tentez de comprendre ce que fait réellement cette fonction, les paramètres qu'elle prend etc. Ensuite seulement, récupérez le fichier shipping.ts contenant le code complet de la fonction et déposez-le dans un dossier `"src/utils/"`.

```
/**
 * Calcule le prix d'une livraison en fonction de la distance,
 *      du poids et du mode d'expédition.
 *
 * @remarks
 * Cette fonction suit les règles métier suivantes :
 * 1. Coût de base (selon la distance) :
 *      - De 0 à 50 km inclus : 10€
 *      - De 51 à 500 km inclus : 25€
 *      - Plus de 500 km : 50€
 *      - Erreur : Une distance négative doit lever une exception.
 *
 * 2. Majoration (selon le poids) :
 *      - Moins de 10 kg : Aucune majoration.
 *      - De 10 à 50 kg inclus : Majoration de 50% sur le coût de base.
 *      - Plus de 50 kg : Livraison impossible (lever une exception).
 *      - Erreur : Un poids négatif ou nul doit lever une exception.
 *
 * 3. Option Express :
 *      - Si le type est 'express', le coût total (après majoration poids)
 *            est multiplié par 2.
 *
 * @param {number} distance - La distance en kilomètres (entier positif).
 * @param {number} weight - Le poids du colis en kilogrammes (entier positif).
 * @param {'standard' | 'express'} type - Le mode de livraison souhaité.
 *
 * @returns {number} Le prix final de la livraison.
 *
 * @throws {Error} "Invalid distance" si la distance est négative.
 * @throws {Error} "Invalid weight" si le poids est <= 0 ou > 50.
 *
 * @example
 * // Cas standard
 * calculateShipping(10, 5, 'standard'); // 10 (Base 10 + 0%)
 *
 * @example
 * // Cas avec majoration poids
 * calculateShipping(100, 20, 'standard'); // 37.5 (Base 25 + 50% = 37.5)
 *
 * @example
 * // Cas express
 * calculateShipping(10, 5, 'express'); // 20 ( (10 + 0%) * 2 )
 */
export function calculateShipping(
    distance: number,
    weight: number,
    type: 'standard' | 'express'
): number {
    // ...
}
```

5.1 Le Catalog-Based Testing

Cette méthode consiste à utiliser un "catalogue" de cas d'erreurs fréquents et de valeurs limites pour chaque paramètre. Par exemple, sur base du catalogue pour un entier qui représente un âge,

```
Normal :            25
Bordure inf :       0, 1
Bordure sup :       120
Hors limite :       -1, 150
Type incorrect :    25.5, null, "trente"
```

Pour les types string et number, n'hésitez pas à jeter un oeil au catalogue de valeurs pour JS dans la table 1 et la table 2.

5.1.1 Exercice

Commencez par décider les différents sets de tests que vous allez implémenter. Pour ce faire, définissez un tableau de tableau qui comprendra tous les sets de tests que vous allez couvrir. Pour exemple, voici une des lignes de cas de test défini par le prof pour couvrir les distances :

👨‍🏫 Informez-vous sur la méthode test.each() de Jest afin d'itérer sur un Array d'Array et de dynamiquement créer l'ensemble des cas de tests, cela vous servira a alléger votre code. Si vous écrivez chaque cas de test manuellement, ce n'est pas un problème.

```
const distanceCases = [
  // [Dist, Poids, Type, Attendu, Description]
  [0, 5, "standard", 10, "Limite basse tranche 1 (0km)"],
```

FIGURE 2 – Données pour un test case

En vous basant sur la figure 4, vérifiez si les tests que vous avez décidés d'implémentés rentrent dans les résultats suivants.

👨‍🏫 Vous devez normalement avoir 12 cas de test. Vous comprendrez pourquoi au point suivant.

```
PASS ./shipping.test.ts
  Shipping Calculator - Tests Fonctionnels
    1. Catalog & Boundaries
      √ Distance 0 km -> Prix 5€ (standard) (3 ms)
      √ Distance 50 km -> Prix 5€ (standard)
      √ Distance 51 km -> Prix 5€ (standard)
      √ Distance 500 km -> Prix 5€ (standard)
      √ Distance 501 km -> Prix 5€ (standard) (1 ms)
      √ Poids 10 kg -> Prix 9€ (standard)
      √ Poids 10 kg -> Prix 10€ (standard) (1 ms)
      √ Poids 10 kg -> Prix 50€ (standard) (1 ms)
      √ Entrée invalide (-1, 5) doit lever "standard" (16 ms)
      √ Entrée invalide (10, 0) doit lever "standard" (1 ms)
      √ Entrée invalide (10, -5) doit lever "standard" (1 ms)
      √ Entrée invalide (10, 51) doit lever "standard" (1 ms)
```

FIGURE 3 – Résultat de l'exécution des tests CBT attendu

5.2 Le N-Wise Testing

Si une fonction prend 3 paramètres ayant chacun 4 valeurs possibles, cela fait $4 \times 4 \times 4 = 64$ tests. Trop long. Le Pairwise (2-wise) testing part du principe que la majorité des bugs sont causés par l'interaction de deux paramètres. On réduit donc drastiquement le nombre de tests en s'assurant que chaque paire de valeurs est testée au moins une fois.

Pour appliquer la méthode N-Wise, nous devons d'abord identifier les valeurs possibles pour chaque paramètre. Nous nous concentrons ici sur les cas valides pour tester la logique de calcul.

- **Distance (3 états) :**
  - D_1 : Courte ($0 - 50$ km)
  - D_2 : Moyenne ($51 - 500$ km)
  - D_3 : Longue (> 500 km)
- **Poids (2 états) :**
  - W_1 : Léger (< 10 kg) $\rightarrow$ Majoration 0%
  - W_2 : Lourd ($10 - 50$ kg) $\rightarrow$ Majoration 50%
- **Type (2 états) :**
  - T_1 : Standard $\rightarrow$ Multiplicateur x1
  - T_2 : Express $\rightarrow$ Multiplicateur x2

Calcul des combinaisons exhaustives : $3 \times 2 \times 2 = 12$ scénarios possibles.

Ensuite, l'objectif du Pairwise est de s'assurer que chaque **paire** de paramètres est testée au moins une fois (ex : D_1 avec W_2, D_1 avec T_1, etc.), sans forcément tester toutes les combinaisons de trois. Cela nous permet de réduire le nombre de tests de 12 à **6 scénarios** tout en garantissant une couverture structurelle forte.

ID	Distance (D)	Poids (W)	Type (T)	Justification des Paires (Exemples)
1	D_1 (Courte)	W_1 (Léger)	T_1 (Standard)	Couvre la paire de base (D_1, W_1) et (W_1, T_1).
2	D_1 (Courte)	W_2 (Lourd)	T_2 (Express)	Teste la majoration poids ET le type Express sur une courte distance. Couvre (D_1, W_2) et (W_2, T_2).
3	D_2 (Moyenne)	W_1 (Léger)	T_2 (Express)	Vérifie l'Express sans majoration poids sur distance moyenne. Couvre (D_2, T_2).
4	D_2 (Moyenne)	W_2 (Lourd)	T_1 (Standard)	Vérifie la majoration poids sur distance moyenne en standard. Couvre (D_2, W_2).
5	D_3 (Longue)	W_1 (Léger)	T_2 (Express)	Vérifie le tarif max avec multiplicateur Express. Couvre (D_3, T_2) et (D_3, W_1).
6	D_3 (Longue)	W_2 (Lourd)	T_1 (Standard)	Vérifie le tarif max avec majoration poids. Couvre (D_3, W_2) et (D_3, T_1).

```
2. Pairwise Combinations
  √ Scénario : 100 km, 20 kg, express -> Total 75 €
  √ Scénario : 600 km, 5 kg, express -> Total 100 € (1 ms)
  √ Scénario : 25 km, 40 kg, standard -> Total 15 € (1 ms)
  √ Scénario : 200 km, 2 kg, standard -> Total 25 € (1 ms)
```

FIGURE 4 – Résultat de l'exécution d'un partie des tests N-Wise attendu

5.3 Conclusion

Rendez-vous compte de l'optimisation réalisée sur la quantité des cas de test. On passe de $4 \times 4 \times 2 = 32$ cas de test possibles à 12 grâce au Catalog based testing pour ensuite terminer à 6 grâce au N-Wise testing.

6 Partie 2 : Tests Structurels (White Box)

Ici, nous avons accès au code. L'objectif est d'atteindre 100% de Branch Coverage (passer dans tous les if et else et un minimum de fois dans les boucles).

6.1 La couverture de code

Jest peut analyser votre code et vous dire quelles lignes n'ont jamais été exécutées lors des tests. C'est l'indicateur principal du test structurel.

6.2 Exercice : Le validateur de mot de passe

Récupérez le fichier password.ts et copiez-le dans "src/utils/password.ts". Créez un fichier "password.test.ts" dans le dossier de tests

```
import { validatePassword } from "../utils/password";

describe("Password Validator - White Box Testing", () => {
  // Test initial pour initialiser le rapport de couverture
  // Ce test ne couvre que la première ligne de la fonction (Branch 1)
  it("devrait rejeter un mot de passe vide", () => {
    const result = validatePassword("", 25);
    expect(result).toBe(false);
  });

  // TODO: Ajoutez vos tests ici pour atteindre 100% de couverture...
});
```

et lancez ensuite la commande de couverture `npm run test:coverage`.

Cette commande a généré un rapport HTML dans "/coverage/lcov-report/index.html". Vous verrez beaucoup de rouge, ce qui est logique vu que rien n'est réellement couvert.

FIGURE 5 – Fichier .html du coverage initial

Écrivez une suite de tests pour faire passer ce fichier à 100% de couverture. Pour ce faire, commencer par identifier les différents cas de tests nécessaire pour ensuite les implémenter.

Vous devrez écrire un test spécifique pour entrer dans chaque if et chaque else. Vous devrez écrire approximativement 13 tests au total.

7 Travail à rendre (À remettre dans 2 semaines)

Pour clôturer ce chapitre sur la qualité logicielle, vous allez devoir appliquer l'ensemble des concepts vus aujourd'hui (Catalog-Based Testing, N-Wise, et Code Coverage) sur une nouvelle fonctionnalité de votre API.

7.1 Le validateur d'inscription

Dans le cadre de votre API, vous souhaitez vérifier les données d'un nouvel utilisateur avant de l'enregistrer en base de données.

Voici les spécifications de la fonction `validateUserRegistration(age, role, email)` :

- **Âge (`number`)** :
  - Doit être un nombre valide.
  - Si inférieur à 18 ans : Inscription refusée (retourne `false`), *sauf* si le rôle est "stagiaire" (retourne `true`).
  - Si supérieur à 120 ans : Doit lever une erreur (`throw new Error("Âge invalide")`).
- **Rôle (`string`)** :
  - N'accepte que trois valeurs exactes : "admin", "user", ou "stagiaire".
  - Toute autre valeur lève une erreur (`throw new Error("Rôle invalide")`).
- **Email (`string`)** :
  - Doit obligatoirement contenir un caractère `@` et un point `.`.
  - Sinon, retourne `false`.

👨‍💻 Lorsque vous implémentez la fonction, n'oubliez pas d'exploiter la force de typage proposée par TypeScript !

Votre mission :

1. **Black Box Testing :** Avant même d'écrire la fonction, utilisez la méthode du *Catalog-Based Testing* pour identifier les valeurs limites et les cas d'erreur de ces trois paramètres. Aidez-vous des tableaux de référence fournis en annexe du TP. Appliquez une logique *Pairwise* pour réduire le nombre de combinaisons.
2. **Implémentation :** Codez cette fonction dans un fichier `src/utils/userValidator.ts`.
3. **White Box Testing :** Écrivez la suite de tests dans `src/tests/userValidator.test.ts` afin d'atteindre **100% de Branch Coverage**.

7.2 Livrables attendus

Dans deux semaines, vous devrez soumettre un rapport au format PDF contenant :

1. **La matrice de tests :** Un tableau clair (similaire à ceux réalisés pour l'exercice du calculateur de livraison) listant vos cas de tests pour le validateur d'inscription, en justifiant vos choix de valeurs limites.
2. **Preuve de réussite :** Une capture d'écran de votre terminal montrant l'exécution de vos tests avec Jest au vert (`PASS`).
3. **Couverture de code :** Une capture d'écran du rapport HTML généré par la commande `--coverage`, prouvant que votre fichier `userValidator.ts` atteint les 100% sur les lignes, les fonctions et surtout les branches.

ℹ️ Vous êtes libre d'utiliser l'éditeur de texte que vous souhaitez, mais un rapport Latex reste toujours plus agréable à lire 😄

Table 1 – Catalogue de tests pour le type `number`

Catégorie	**Valeur TypeScript**	**Objectif / Risque**
Zéro	`0`, `-0`	Division par zéro, erreurs de signe.
Limites Sûres	`Number.MAX_SAFE_INTEGER`	Précision des entiers $> 2^{53} - 1$.
Dépassement	`Infinity`, `-Infinity`	Gestion des calculs hors limites.
Précision	`0.1 + 0.2`	Erreurs d'arrondi binaire (IEEE 754).
Invalide	`NaN`	Propagation d'erreurs d'opérations illégales.
Nullables	`null`, `undefined`	Absence de donnée ou typage lâche.

Table 2 – Catalogue de tests pour le type `string`

Catégorie	**Valeur TypeScript**	**Objectif / Risque**
Vide	`""`	Validation des champs obligatoires.
Blancs	`" "`, `"\n\t"`	Échec du *trimming* ou validation d'espace.
Injection	`"<script>...</script>"`	Faille XSS ou injection de code.
Unicode	`" "`, `" "`	Support UTF-8 et caractères multi-octets.
Longueur	`"a".repeat(10000)`	Dépassement de capacité ou lenteur UI.
Coercition	`"123"`, `"true"`	Confusion entre types primitifs et chaînes.